Team skywalker: a learned win-probability ensemble on top of a heuristic planner

2-player Catan agent for the Hyde hackathon, 17 October 2026

1. Summary

Our final agent combines two parts. A hand-written planner (built by our teammates, inspired by F) proposes every move. A small neural network, trained on our own self-play games to predict the probability of winning, checks the planner's choice during the main turn and overrules it when it is clearly better. The network ended up as an **ensemble of 10 models**. Over the day we went from F's own level (49% against F) to **80-83% against F** in our local tests, and 0.81 against F and 0.66 against O on the public leaderboard. The single biggest lesson: a value function learned from one player's games is only reliable on the positions that player visits.

Design we submitted (public leaderboard, 1,000 games)	vs F	vs O	What we learned
The network makes most decisions; hand-made rules only for opening, robber and knight	0.801 ±0.025	0.612 ±0.030	Strong against F, weaker against O
The hand-made planner makes every decision; the network only overrules clear mistakes	0.810 ±0.024	0.663 ±0.029	Same against F, +5 points against O: rules leading is more robust
As above, with a stronger preference for Longest Road	0.812 ±0.024	0.659 ±0.029	No change: small weight tweaks sit below the noise

2. The problem in one paragraph

Two-player Catan, first to 10 victory points, no trading between players. The only hidden information is the opponent's development cards; we see their resource hand exactly. Each agent gets about 12.5 ms of CPU per decision (300 ms start, at most 1.5 s per game), on a machine about twice as slow as our laptop under load. The anchor bot **F** looks one move ahead and scores positions with 13 hand-set weights; **O** is a stronger trained bot we never had access to. With roughly 6× F's compute, the question was how to spend it well.

3. How the agent decides

Learned value. We describe a position with 102 numbers, computed only from what our seat can see: production per resource for both players (with the robber removed), settlements, cities, roads, ports, cards in hand and whether they afford a city/settlement/dev card, how many good building spots are reachable with 0, 1 or 2 more roads, Longest Road and Largest Army status, victory points (the opponent's hidden points are estimated from the unseen deck), and whose turn it is. A small network ($102 \to 128 \to 64 \to 1$) learns P(win) from final game results, in the spirit of Buro's Othello evaluation and TD-Gammon.

Search. For each legal move we build the resulting position and score it. Random events are averaged exactly instead of sampled: all 11 dice totals, every possible development card, every possible stolen card. Moves that win immediately are always taken. If the move keeps the turn with us (e.g. a bank trade), we also look one move further (“trade, then build”) and keep the best. A faster game copy (no pickling; verified to replay 40 full games identically) keeps the cost around 1-3 ms per decision.

Combination. The teammates' planner plays the opening (it simulates F's replies to pick settlements, with a learned opening score), robber and knight decisions, and proposes the main-turn move. Our ensemble overrides it only when it believes another move is at least 3 points of win probability better, and only while 0.25 s of clock remains. In the last version the robber always targets the resources the opponent needs for its next city or settlement.

4. The journey: key decisions and what we measured

All local numbers are win rates against F through the official harness and seeds, ± is the 95% interval. Every row is logged in our experiment file.

Step	Change	vs F	Why it mattered
1	Network trained on 10,000 F-vs-F games, play greedily	10.5% (400 g)	Failed: it bought dev cards in 232 spots where F ends its turn; F rarely does that, so the network had never seen the result
2	Add 5,000 games played by step 1 vs F, retrain	55.0% (400 g)	Policy iteration: train on the positions you actually reach
3	Same-turn look-ahead (trade then build)	56.3% vs 49.2%	+7 points at the same network
4	Average 3 networks trained with different seeds	67.8% vs 59.1% (1,000 g)	Biggest single gain: less noise when picking the best of many moves
5	5 networks; more policy-iteration rounds	70.1%, then 68.3%	Iteration plateaued
6	Borrow teammates' opening, robber and knight rules	78.5% vs 67.9% (1,000 g)	+10.6: hand-made rules beat the network where its data is thin
7	Planner leads, network only overrides	80.6% (1,000 g)	Better against O on the leaderboard
8	10-model ensemble from the two latest training rounds	82.1% vs 80.6%	Same direction vs our O stand-in: 62.7% vs 61.2%
9	Robber always denies the opponent's next build	83.0% vs 82.1%	Within noise; kept, not worse

5. Studying our opponents

Head-to-head breakdowns. We wrote a tool that plays two agents and reports, for winners and losers separately, settlements, cities, Longest Road, Largest Army, dev cards bought and production. Against our teammates' heuristic bot, our network agent lost when that bot “went wide”: 3.5 settlements, Longest Road in 83% and Largest Army in 71% of its wins, while we held Largest Army in only 9% of our losses. Opening quality was equal (21.7 vs 21.9 pips), so the opening was not the cause.

A symptom is not a cause. Winners buy more dev cards, so we tried a bonus for buying them. It changed nothing (50.0% and 48.7% against the heuristic bot, 69.3% and 67.0% vs F). Winners buy more cards simply because they have more resources. Training on 5,000 games against that bot also moved us only from 49.7% to 52.0%. What worked was taking its rules for the opening, robber and knight directly.

Why F still beats us. In the ~18% of games F wins, it builds 3 cities and reaches 83 pips of production versus our 42, from equal openings. These are mostly dice-driven snowballs; a robber that always targets F's next build was our last, small response.

An O stand-in. We could not play O, so we used our pure network bot as a proxy for “a trained value bot”. It ranked our three scored submissions in the same order as the leaderboard's O column (the planner-led design clearly best), so we tuned against a pool of F, the stand-in and the teammates' bot instead of F alone.

6. What did not work

- Greedy play on a value learned only from F's games (10.5%); see step 1.
- Training a large network without regularisation: validation loss rose after one epoch, because two games per board let it memorise the board. Fix: dropout, early stopping, and a validation split by game.

- Override margin 0 or 0.06 instead of 0.03: worse against the pool (e.g. 51.3% and 49.0% vs 57.3% against the teammates' bot).
- Longest Road and dev-card weight tweaks: looked better at 400 games, identical at 1,000 (61.0% vs 61.2%).
- 15-model ensemble with older networks: worse than 10 (80.9% vs 82.1%).
- Further policy-iteration rounds once the planner led: 80.1%, no gain, because the network rarely overrides.

7. Lessons that hold beyond Catan

- **A learned value is only trustworthy where its data lives.** Acting on it moves you off that distribution; retrain on your own games (policy iteration / DAgger).
- **When you pick the best of many noisy scores, the noise wins.** Ensembles reduce that “winner's curse” and gave our largest jump.
- **Sample size decides.** 300 games give ±5.7 points. Several “84%” results fell back to 80% at 1,000 games, so we only accepted changes measured on 1,000 paired games.
- **Rules and learning complement each other.** Hand-made rules were better where data was thin (opening, robber); the network was better at weighing ordinary trade-offs.
- **Measure CPU on the slow machine.** The leaderboard charged 1.3-1.5× our local CPU and up to 1.25 s in the worst game against a 1.5 s cap, so we never added heavier search.

8. Reproducibility and compute

Everything ran on one Apple M3 laptop (8 cores). About 60,000 self-play games produced roughly 9 million labelled positions; each network trains in about one minute with PyTorch and runs in plain numpy at play time. Training seeds never overlap the evaluation seeds. Scripts: data generation, training, a parallel copy of the official evaluation, the game-breakdown tool and the pool tuner.

References and public material

catanatron by Bryan Collazo (GPL-3.0), the pinned engine and the F bot; the organisers' starter kit; PyTorch and numpy. M. Buro, Improving heuristic mini-max search by supervised learning, AI 2002. G. Tesauro, TD-Gammon, CACM 1995. S. Ross, G. Gordon, D. Bagnell, DAgger, AISTATS 2011. The planner, opening model and heuristic rules are our teammates' work within team skywalker. A language model (Claude) helped write code and analyse results, as the rules allow.